TelemetryDiagnosticControls.java (clase refactorizada)

TelemetryDiagnosticControls.java (clase refactorizada)

```java
package tddmicroexercises.telemetrysystem;

public class TelemetryDiagnosticControls
{
    private static final String DIAGNOSTIC_CHANNEL_CONNECTION_STRING = "*111#";
    private static final int MAX_CONNECTION_ATTEMPTS = 3;

    private final TelemetryClient telemetryClient;
    private String diagnosticInfo = "";

    public TelemetryDiagnosticControls()
    {
        this(new TelemetryClient());
    }

    // Inyección de dependencias (DIP): permite sustituir el cliente por un mock en los tests.
    public TelemetryDiagnosticControls(TelemetryClient telemetryClient)
    {
        this.telemetryClient = telemetryClient;
    }

    public String getDiagnosticInfo()
    {
        return diagnosticInfo;
    }

    public void setDiagnosticInfo(String diagnosticInfo)
    {
        this.diagnosticInfo = diagnosticInfo;
    }

    public void checkTransmission() throws Exception
    {
        diagnosticInfo = "";

        telemetryClient.disconnect();

        int retryLeft = MAX_CONNECTION_ATTEMPTS;
        while (!telemetryClient.getOnlineStatus() && retryLeft > 0)
        {
            telemetryClient.connect(DIAGNOSTIC_CHANNEL_CONNECTION_STRING);
            retryLeft -= 1;
        }

        if (!telemetryClient.getOnlineStatus())
        {
            throw new Exception("Unable to connect.");
        }

        telemetryClient.send(TelemetryClient.DIAGNOSTIC_MESSAGE);
        diagnosticInfo = telemetryClient.receive();
    }
}
```

TelemetryDiagnosticControlsTest.java (tests con Mockito) (parte 1)

TelemetryDiagnosticControls.java (clase refactorizada)

```java
package tddmicroexercises.telemetrysystem;

public class TelemetryDiagnosticControls
{
    private static final String DIAGNOSTIC_CHANNEL_CONNECTION_STRING = "*111#";
    private static final int MAX_CONNECTION_ATTEMPTS = 3;

    private final TelemetryClient telemetryClient;
    private String diagnosticInfo = "";

    public TelemetryDiagnosticControls()
    {
        this(new TelemetryClient());
    }

    // Inyección de dependencias (DIP): permite sustituir el cliente por un mock en los tests.
    public TelemetryDiagnosticControls(TelemetryClient telemetryClient)
    {
        this.telemetryClient = telemetryClient;
    }

    public String getDiagnosticInfo()
    {
        return diagnosticInfo;
    }

    public void setDiagnosticInfo(String diagnosticInfo)
    {
        this.diagnosticInfo = diagnosticInfo;
    }

    public void checkTransmission() throws Exception
    {
        diagnosticInfo = "";

        telemetryClient.disconnect();

        int retryLeft = MAX_CONNECTION_ATTEMPTS;
        while (!telemetryClient.getOnlineStatus() && retryLeft > 0)
        {
            telemetryClient.connect(DIAGNOSTIC_CHANNEL_CONNECTION_STRING);
            retryLeft -= 1;
        }

        if (!telemetryClient.getOnlineStatus())
        {
            throw new Exception("Unable to connect.");
        }

        telemetryClient.send(TelemetryClient.DIAGNOSTIC_MESSAGE);
        diagnosticInfo = telemetryClient.receive();
    }
}
```

TelemetryDiagnosticControlsTest.java (tests con Mockito) (parte 2)

```
    }

    @Test
    public void CheckTransmission_should_clear_previous_diagnostic_info_when_connection_fails() throws Exception
    {
        controls.setDiagnosticInfo("old info");
        when(telemetryClient.getOnlineStatus()).thenReturn(false);

        assertThrows(Exception.class, () -> controls.checkTransmission());

        assertEquals("", controls.getDiagnosticInfo());
    }

    @Test
    public void CheckTransmission_should_connect_only_once_when_first_attempt_succeeds() throws Exception
    {
        when(telemetryClient.getOnlineStatus()).thenReturn(false, true);

        controls.checkTransmission();

        verify(telemetryClient, times(1)).connect(CONNECTION_STRING);
    }

    @Test
    public void CheckTransmission_should_not_connect_if_client_is_already_online_after_disconnect() throws Exception
    {
        when(telemetryClient.getOnlineStatus()).thenReturn(true);
        when(telemetryClient.receive()).thenReturn(DIAGNOSTIC_RESPONSE);

        controls.checkTransmission();

        verify(telemetryClient, never()).connect(org.mockito.ArgumentMatchers.anyString());
        assertEquals(DIAGNOSTIC_RESPONSE, controls.getDiagnosticInfo());
    }

    @Test
    public void DiagnosticInfo_should_be_empty_by_default()
    {
        assertEquals("", controls.getDiagnosticInfo());
    }

    @Test
    public void SetDiagnosticInfo_should_store_the_given_value()
    {
        controls.setDiagnosticInfo("custom");

        assertEquals("custom", controls.getDiagnosticInfo());
    }
}
```

Resultado de mvn test

TelemetryDiagnosticControls.java (clase refactorizada)

```java
package tddmicroexercises.telemetrysystem;

public class TelemetryDiagnosticControls
{
    private static final String DIAGNOSTIC_CHANNEL_CONNECTION_STRING = "*111#";
    private static final int MAX_CONNECTION_ATTEMPTS = 3;

    private final TelemetryClient telemetryClient;
    private String diagnosticInfo = "";

    public TelemetryDiagnosticControls()
    {
        this(new TelemetryClient());
    }

    // Inyección de dependencias (DIP): permite sustituir el cliente por un mock en los tests.
    public TelemetryDiagnosticControls(TelemetryClient telemetryClient)
    {
        this.telemetryClient = telemetryClient;
    }

    public String getDiagnosticInfo()
    {
        return diagnosticInfo;
    }

    public void setDiagnosticInfo(String diagnosticInfo)
    {
        this.diagnosticInfo = diagnosticInfo;
    }

    public void checkTransmission() throws Exception
    {
        diagnosticInfo = "";

        telemetryClient.disconnect();

        int retryLeft = MAX_CONNECTION_ATTEMPTS;
        while (!telemetryClient.getOnlineStatus() && retryLeft > 0)
        {
            telemetryClient.connect(DIAGNOSTIC_CHANNEL_CONNECTION_STRING);
            retryLeft -= 1;
        }

        if (!telemetryClient.getOnlineStatus())
        {
            throw new Exception("Unable to connect.");
        }

        telemetryClient.send(TelemetryClient.DIAGNOSTIC_MESSAGE);
        diagnosticInfo = telemetryClient.receive();
    }
}
```